Kafka Internals

Objectives

- Cluster membership
- Controller
- Replication
- Request processing
- Physical storage

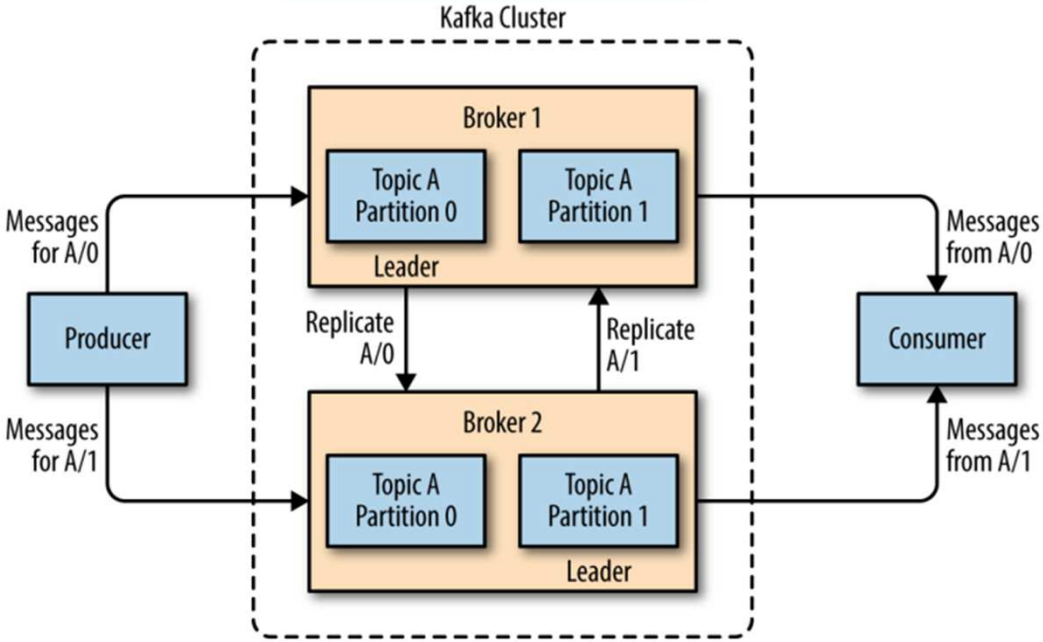
Cluster Membership

- Kafka uses Apache Zookeeper to maintain the list of brokers that are currently members of a cluster
- Every broker has a unique identifier
- Every time a broker process starts, it registers itself with its ID in Zookeeper by creating an ephemeral node
- Different Kafka components subscribe to the /brokers/ids path in Zookeeper where brokers are registered so they get notified when brokers are added or removed.
- If you completely lose a broker and start a brand new broker with the ID of the old one, it will immediately join the cluster in place of the missing broker with the same partitions and topics assigned to it.

Controller

- The controller is one of the Kafka brokers that, in addition to the usual broker functionality, is responsible for electing partition leaders.
- The first broker that starts in the cluster becomes the controller by creating an ephemeral node in ZooKeeper called /controller
- When the controller broker is stopped or loses connectivity to Zookeeper, the ephemeral node will disappear, and other brokers will get informed. One of next available broker becomes the controller.

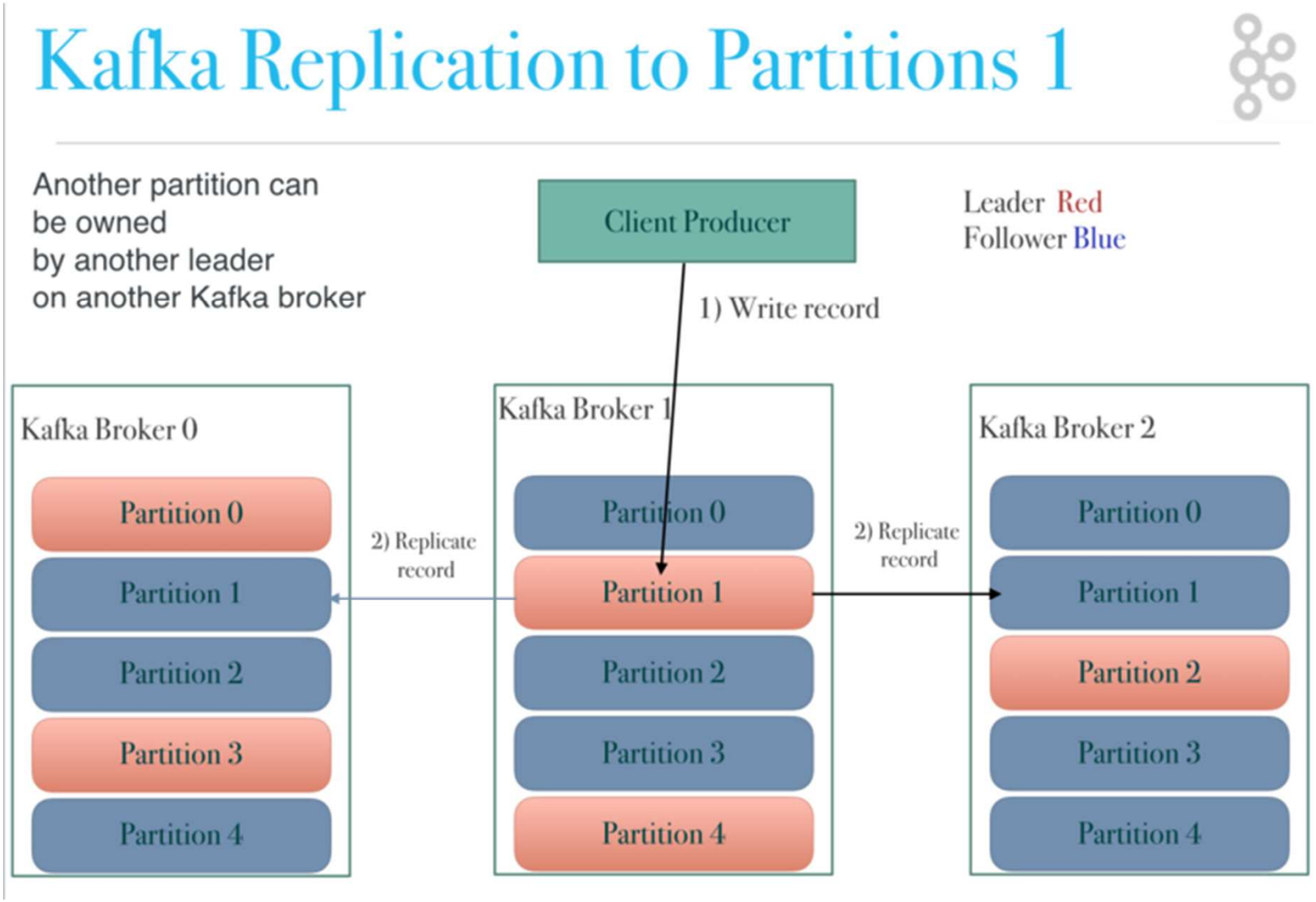
Replication



- Replication is at the heart of Kafka's architecture.
- Replication is critical because it is the way Kafka guarantees availability and durability when individual nodes inevitably fail.
- Each topic is partitioned, and each partition can have multiple replicas.
- A partition may be assigned to multiple brokers, which will result in the partition being replicated.
- Replication provides broker (leader) failure, as another broker can take leadership, but only in single cluster.
- A partition is owned by a single broker in the cluster, and that broker is called the **leader** of the partition.
- A partition may be assigned to multiple brokers, which will result in the partition being replicated providing provides redundancy of messages in the partition, this way broker can take over leadership if there is a broker failure.

Replication

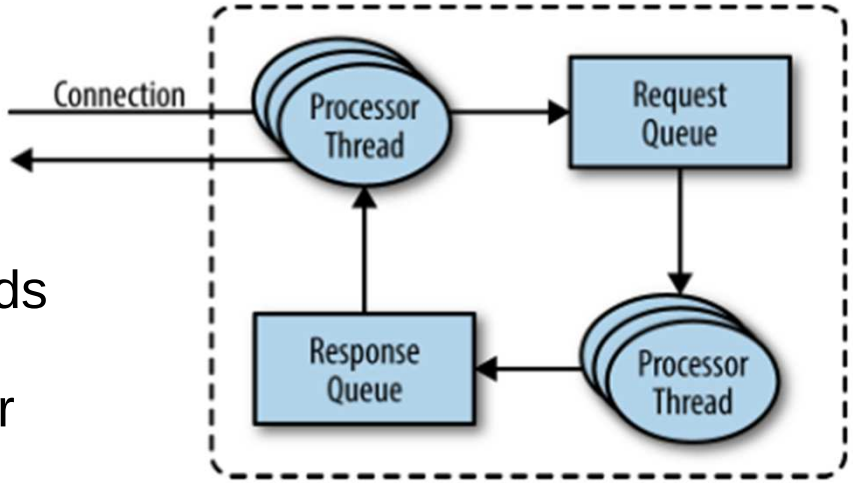
http://cloudurable.com/images/kafka-architecture-topics-replication-to-partition-1.png



Practice

- Follow the steps from Replication Practice

Request Processing



- Clients always initiate connections and send requests, and the broker processes the requests and responds to them
- All requests have a standard header that includes: Request type (also called API key), Request version, Correlation ID – a number that uniquely identifies the request, Client ID – used to identify the application that sent the request
- For each port the broker listens on,
  - The broker runs an acceptor thread that creates a connection and hands it over to a processor thread (also called network threads) for handling.
  - The network threads are responsible for taking requests from client connections, placing them in a request queue.
  - The network threads are also responsible for picking up responses from a response queue and sending them back to clients
- Types of Requests – Both are sent to Leader
  - Produce requests – Sent by producers
  - Fetch requests – Sent by Consumers

Request Processing

- If produce or fetch request of a partition does not have a leader then Producer or Consumer will get “Not a Leader for Partition” error.
  - To solve above issue, Kafka clients use another request type called a metadata request, which includes a list of topics the client is interested in.
- Clients typically cache this information and use it to direct produce and fetch requests to the correct broker for each partition, and can be controlled by ‘metadata.max.age.ms’ configuration property.
- Client cache is also updated if a client receives the “Not a Leader” error to one of its requests, it will refresh its metadata before trying to send the request again

Physical Storage

- Partition Allocation
  - When you create a topic, Kafka first decides how to allocate the partitions between brokers. Suppose you have 6 brokers and you decide to create a topic with 10 partitions and a replication factor of 3. Kafka now has 30 partition replicas to allocate to 6 brokers.
- File Management
  - Retention is an important concept in Kafka, hence to find the messages that need purging Kafka splits each partition into segments.
  - The segment Kafka currently writing to is called an active segment, which is never deleted.
  - Can be found at log.dirs location
- File Format
  - Each segment is stored in a single data file. Inside the file, Kafka messages and their offsets are stored.
    - Using ‘bin/kafka-run-class.sh kafka.tools.DumpLogSegments’ tool contents can be examined.
- Indexes
  - Kafka allows consumers to start fetching messages from any available offset.
  - The index maps offsets to segment files and positions within the file.

Physical Storage

- Compaction
  - Log compaction is a mechanism to give finer-grained per-record retention, rather than the coarser-grained time-based retention.
  - The idea is to selectively remove records where we have a more recent update with the same primary key
  - It addresses use cases and scenarios such as restoring state after application crashes or system failure, or reloading caches after application restarts during operational maintenance.
  - Use ‘cleanup.policy’ topic level configuration which is ‘delete’ or ‘compact’
    - Delete – Designates the retention policy (time/size)to use on old log segments.
    - Compact – The "compact" setting will enable log compaction on the topic.
- Deleted Events
  - In order to delete a key from the system completely, not even saving the last message, the application must produce a message that contains that key and a null value (known as a tombstone).
  - The cleaner thread will remove the tombstone message, and the key will be gone from the partition in Kafka.

Summary

- Cluster membership
- Controller
- Replication
- Request processing
- Physical storage

Practice

- This practice teaches